

Vector vs. diccionario para mapas pequeños

Eduardo Alanis García

2026-08-26

Table of contents

1 Actividad: Vector vs. diccionario para mapas pequeños	2
1.1 Introducción	2
1.2 Implementación	2
1.2.1 Valor centinela	2
1.2.2 Vector de capacidad fija	2
1.2.3 Implementación de <code>buscar</code> para el vector	3
1.2.4 Implementación de <code>insertar!</code> para el vector	3
1.2.5 Implementación de <code>borrar!</code> para el vector	4
1.3 Implementación del diccionario	4
1.4 Preparación de los datos	5
1.5 Comprobación básica de las funciones	6
2 Experimentos	6
2.1 Tamaños de entrada	6
2.2 Experimento: <code>buscar</code>	7
2.2.1 Resultados de <code>buscar</code>	8
2.2.2 Análisis de <code>buscar</code>	10
2.3 Experimento: <code>borrar!</code>	11
2.3.1 Resultados de <code>borrar!</code>	12
2.3.2 Análisis de <code>borrar!</code>	14
2.4 Experimento: <code>insertar!</code>	14
2.4.1 Resultados de <code>insertar!</code>	15
2.4.2 Análisis de <code>insertar!</code>	17
3 Comparación de rangos	17
4 Discusión	18
5 Conclusiones	18

1 Actividad: Vector vs. diccionario para mapas pequeños

1.1 Introducción

En esta actividad se compara el desempeño de dos estructuras de datos para representar un mapa de llaves enteras (`Int64`) a valores flotantes (`Float64`). La primera estructura es un vector de pares sin ordenar, en el que las operaciones se realizan mediante un recorrido lineal. La segunda estructura es un `Dict{Int64,Float64}`, que utiliza una tabla hash.

El objetivo es comparar experimentalmente las operaciones `buscar`, `insertar!` y `borrar!` para diferentes tamaños de entrada. De esta manera, los resultados experimentales permiten observar el comportamiento de cada estructura en una computadora real.

a

1.2 Implementación

1.2.1 Valor centinela

Se utilizará una llave que no forme parte de los datos reales:

```
CENTINELA = typemin{Int64}
```

-9223372036854775808

1.2.2 Vector de capacidad fija

El vector se crea con una capacidad determinada desde el inicio. Esto permite mantener fija su longitud durante las operaciones y evita utilizar `push!`, `pop!` o `deleteat!`.

```
function crear_vector(capacidad)
    return Vector{Pair{Int64,Float64}}([CENTINELA => 0.0 for i in 1:capacidad])
end
```

`crear_vector` (generic function with 1 method)

1.2.3 Implementación de buscar para el vector

```
function buscar_vector(vector, llave)
  for par in vector
    if par.first != CENTINELA && par.first == llave
      return par.second
    end
  end
  return nothing
end
```

buscar_vector (generic function with 1 method)

Si la llave no se encuentra, devuelve nothing.

1.2.4 Implementación de insertar! para el vector

```
function insertar_vector!(v, llave, valor)
  for i in 1:length(v)

    # Si la llave ya existe, se actualiza el valor
    if v[i].first == llave
      v[i] = llave => valor
      return v
    end

    # Si no existe, busca una posición marcada como vacía para colocar el nuevo par
    if v[i].first == CENTINELA
      v[i] = llave => valor
      return v
    end
  end

  error("No hay ningun espacio disponible en vector")
end
```

insertar_vector! (generic function with 1 method)

Si la llave ya existe, actualiza su valor. Si no existe, utiliza la primera posición marcada como vacía.

1.2.5 Implementación de borrar! para el vector

```
function borrar_vector!(v, llave)
    for i in 1:length(v)
        if v[i].first == llave
            v[i] = CENTINELA => 0.0
            return v
        end
    end

    return v
end
```

borrar_vector! (generic function with 1 method)

La posición encontrada no se elimina. En su lugar, se sobrescribe con el centinela para indicar que está disponible nuevamente.

1.3 Implementación del diccionario

Funciones equivalentes para poder comparar ambas estructuras.

```
function crear_dict_mapa(capacidad::Int)
    d = Dict{Int64,Float64}()
    sizehint!(d, capacidad)
    return d
end

function buscar_dict(d, llave)
    if haskey(d, llave)
        return d[llave]
    end
    return nothing
end

function insertar_dict!(d, llave, valor)
    d[llave] = valor
    return d
end
```

```
function borrar_dict!(d, llave)
    delete!(d, llave)
    return d
end
```

borrar_dict! (generic function with 1 method)

En el caso del Dict, `sizehint!` se utiliza para reservar espacio y reducir el efecto del crecimiento de la tabla durante los experimentos.

1.4 Preparación de los datos

Para cada tamaño `n` se crean ambas estructuras con las mismas llaves y los mismos valores. Las llaves se mezclan para capturar el caso promedio y no solo el mejor o peor caso posicional.

```
using BenchmarkTools
using Random
using DataFrames
using Plots
using Statistics

#Random.seed!(1234)

function preparar_datos(n, capacidad_vector = n)
    v = crear_vector(capacidad_vector)
    d = crear_dict_mapa(n)
    llaves = collect{Int64}(1:n)
    shuffle!(llaves) # Desordena de forma aleatoria, por la indicacion la llave a con
    valores = rand(n)

    for i in 1:n
        insertar_vector!(v, llaves[i], valores[i])
        insertar_dict!(d, llaves[i], valores[i])
    end

    return v, d
end
```

preparar_datos (generic function with 2 methods)

1.5 Comprobación básica de las funciones

Solo vemos que funcionan correctamente.

```
v_prueba, d_prueba = preparar_datos(8, 9)
llave_prueba = 3

println("Resultado del vector: ", buscar_vector(v_prueba, llave_prueba))
println("Resultado del diccionario: ", buscar_dict(d_prueba, llave_prueba))
```

Resultado del vector: 0.027118708941859304
Resultado del diccionario: 0.027118708941859304

```
insertar_vector!(v_prueba, 9, 10.0)
insertar_dict!(d_prueba, 9, 10.0)

println("Resultado del vector: ", buscar_vector(v_prueba, 9))
println("Resultado del diccionario: ", buscar_dict(d_prueba, 9))
```

Resultado del vector: 10.0
Resultado del diccionario: 10.0

```
borrar_vector!(v_prueba, 9)
borrar_dict!(d_prueba, 9)

println("Resultado del vector: ", buscar_vector(v_prueba, 9))
println("Resultado del diccionario: ", buscar_dict(d_prueba, 9))
```

Resultado del vector: nothing
Resultado del diccionario: nothing

2 Experimentos

2.1 Tamaños de entrada

Se utilizarán potencias de 2 comenzando en 2^2 . Vamos a considera $2^{12} = 4096$ como máximo práctico dado que no tarda tanto mi PC.

```
tamaños = 2 .^ (2:12)
tamaños
```

```
11-element Vector{Int64}:
 4
 8
16
32
64
128
256
512
1024
2048
4096
```

2.2 Experimento: buscar

La llave se elige aleatoriamente dentro del `setup=` para cada muestra. De esta manera, no se utiliza siempre la misma posición del vector.

```
resultados_buscar_vector = []
resultados_buscar_dict = []

for n in tamaños
    v, d = preparar_datos(n)

    resultado_vector = @benchmark buscar_vector($v, llave) setup=(
        llave = rand{Int64}(1:$n)
    ) samples=100

    resultado_dict = @benchmark buscar_dict($d, llave) setup=(
        llave = rand{Int64}(1:$n)
    ) samples=100

    push!(resultados_buscar_vector,
        (
            n = n,
            trial = resultado_vector,
            minimo = minimum(resultado_vector).time,
            medio = mean(resultado_vector).time,
```

```

        maximo = maximum(resultado_vector).time
    )
)

push!(resultados_buscar_dict,
    (
        n = n,
        trial = resultado_dict,
        minimo = minimum(resultado_dict).time,
        medio = mean(resultado_dict).time,
        maximo = maximum(resultado_dict).time
    )
)
end

```

2.2.1 Resultados de buscar

```

tabla_buscar = DataFrame(
    n = [r.n for r in resultados_buscar_vector],
    vector_minimo = [r.minimo for r in resultados_buscar_vector],
    vector_medio = [r.medio for r in resultados_buscar_vector],
    vector_maximo = [r.maximo for r in resultados_buscar_vector],
    dict_minimo = [r.minimo for r in resultados_buscar_dict],
    dict_medio = [r.medio for r in resultados_buscar_dict],
    dict_maximo = [r.maximo for r in resultados_buscar_dict]
)

tabla_buscar.comparar = ifelse.(
    tabla_buscar.vector_medio .< tabla_buscar.dict_medio,
    "Vector",
    "Dict"
)

tabla_buscar

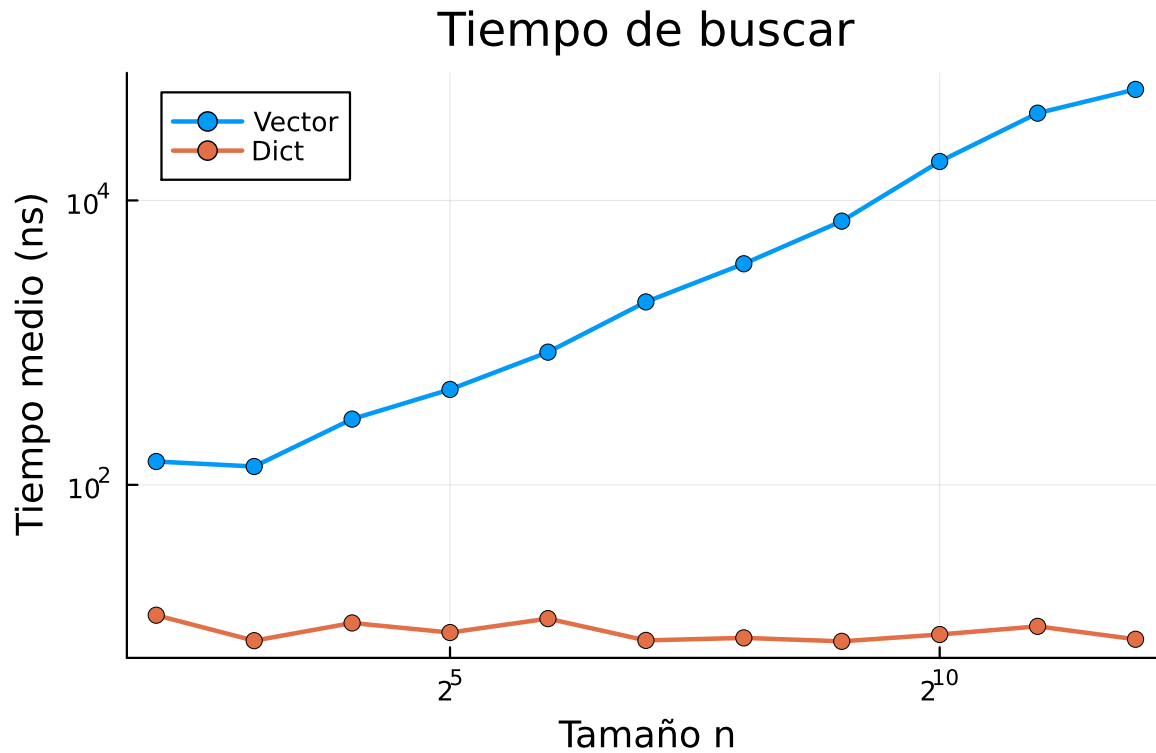
```

	n	vector_minimo	vector_medio	vector_maximo	dict_minimo	dict_medio	dict_maximo
	Int64	Float64	Float64	Float64	Float64	Float64	Float64
1	4	31.2563	145.88	475.276	11.3113	12.1131	13.7137
2	8	30.8853	134.574	233.099	7.2	8.019	12.6
3	16	30.8853	289.924	946.881	7.4	10.683	169.3
4	32	31.8913	468.706	853.823	7.2	9.15	17.2
5	64	57.2435	857.08	1797.89	8.5	11.462	20.1
6	128	62.4748	1932.96	5990.04	7.20721	8.05506	12.2122
7	256	83.7022	3584.73	7122.74	7.2	8.389	18.0
8	512	32.6962	7150.07	21088.7	7.20721	7.93493	18.4184
9	1024	328.024	18780.5	51676.0	7.70771	8.86086	16.9169
10	2048	130.03	41079.1	104036.0	7.2	10.111	156.2
11	4096	34.8837	60363.0	1.70585e5	7.2	8.204	15.2

```
ENV["GKSwstype"] = "100"
using Plots

default(fmt = :png)
```

```
plot(
    tabla_buscar.n,
    [tabla_buscar.vector_medio tabla_buscar.dict_medio],
    label=["Vector" "Dict"],
    xlabel="Tamaño n",
    ylabel="Tiempo medio (ns)",
    title="Tiempo de buscar",
    marker=:circle,
    linewidth=2,
    xscale=:log2,
    yscale=:log10
)
```



```
for fila in eachrow(tabla_buscar)
    println("n=$(fila.n): gana $(fila.comparar)")
end
```

```
n=4: gana Dict
n=8: gana Dict
n=16: gana Dict
n=32: gana Dict
n=64: gana Dict
n=128: gana Dict
n=256: gana Dict
n=512: gana Dict
n=1024: gana Dict
n=2048: gana Dict
n=4096: gana Dict
```

2.2.2 Análisis de buscar

La tabla y la gráfica anterior permiten identificar los tamaños para los cuales cada estructura presenta el menor tiempo medio. Tenemos que en todo momento **Dict** es más rápido que el

vector en todo momento.

2.3 Experimento: borrar!

Como `borrar!` modifica la estructura, cada realizamos una copia de los datos. Además, se utiliza `evals=1` para que cada muestra del benchmark se ejecute una sola vez.

```
resultados_borrar_vector = []
resultados_borrar_dict = []

for n in tamaños
    v, d = preparar_datos(n)

    resultado_vector = @benchmark borrar_vector!(v, llave) setup=(
        v = copy($v);
        llave = rand{Int64}(1):Int64($n))
    ) samples=100 evals=1

    resultado_dict = @benchmark borrar_dict!(d, llave) setup=(
        d = copy($d);
        llave = rand{Int64}(1):Int64($n))
    ) samples=100 evals=1

    push!(resultados_borrar_vector, (
        n = n,
        trial = resultado_vector,
        minimo = minimum(resultado_vector).time,
        medio = mean(resultado_vector).time,
        maximo = maximum(resultado_vector).time
    ))

    push!(resultados_borrar_dict, (
        n = n,
        trial = resultado_dict,
        minimo = minimum(resultado_dict).time,
        medio = mean(resultado_dict).time,
        maximo = maximum(resultado_dict).time
    ))
end
```

2.3.1 Resultados de borrar!

```

tabla_borrar = DataFrame(
    n = [r.n for r in resultados_borrar_vector],
    vector_minimo = [r.minimo for r in resultados_borrar_vector],
    vector_medio = [r.medio for r in resultados_borrar_vector],
    vector_maximo = [r.maximo for r in resultados_borrar_vector],
    dict_minimo = [r.minimo for r in resultados_borrar_dict],
    dict_medio = [r.medio for r in resultados_borrar_dict],
    dict_maximo = [r.maximo for r in resultados_borrar_dict]
)

tabla_borrar.comparar = ifelse.(
    tabla_borrar.vector_medio .< tabla_borrar.dict_medio,
    "Vector",
    "Dict"
)

tabla_borrar

```

	n	vector_minimo	vector_medio	vector_maximo	dict_minimo	dict_medio	dict_maximo
	Int64	Float64	Float64	Float64	Float64	Float64	Float64
1	4	200.0	340.0	5700.0	0.001	50.0005	100.0
2	8	200.0	347.0	5800.0	0.001	53.0005	100.0
3	16	200.0	372.0	5900.0	0.001	49.0005	100.0
4	32	300.0	461.0	7700.0	0.001	68.0003	200.0
5	64	200.0	349.0	5900.0	0.001	45.0006	100.0
6	128	300.0	422.0	6200.0	0.001	52.0005	100.0
7	256	400.0	545.0	7100.0	0.001	71.0003	200.0
8	512	400.0	719.0	8300.0	0.001	106.0	200.0
9	1024	300.0	677.0	6300.0	0.001	73.0003	400.0
10	2048	400.0	1096.0	6200.0	0.001	93.0002	300.0
11	4096	400.0	1644.0	7400.0	0.001	86.0002	300.0

```

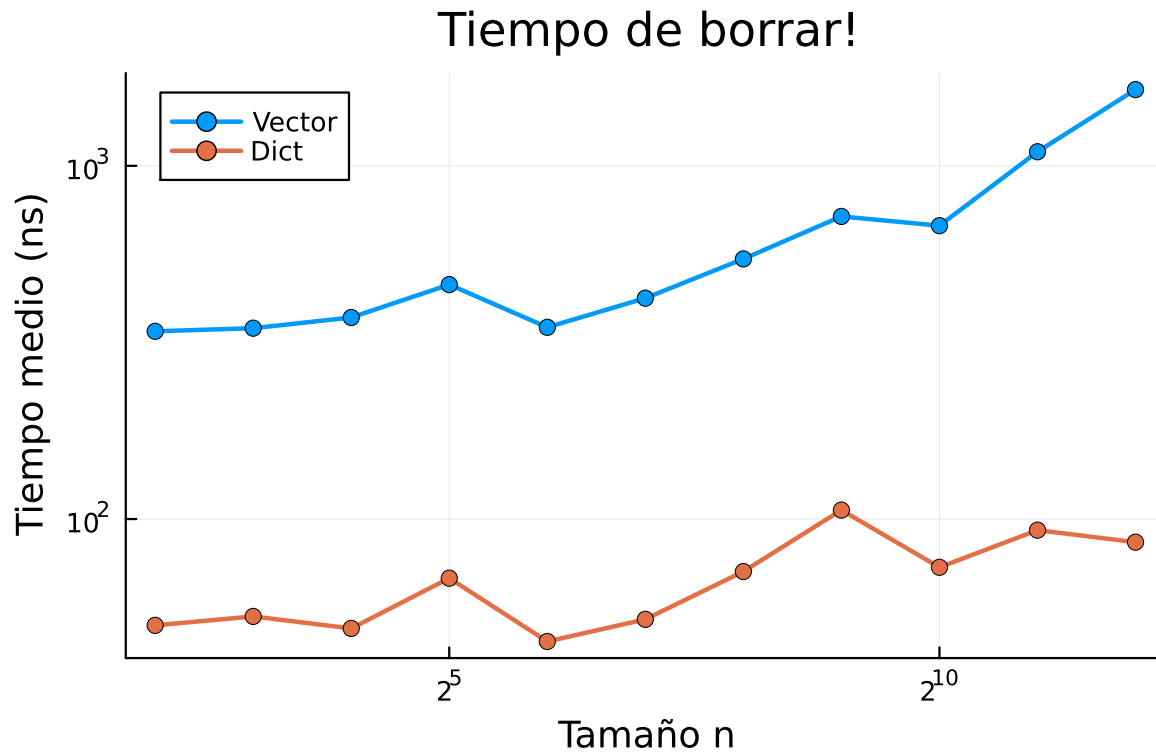
plot(
    tabla_borrar.n,
    [tabla_borrar.vector_medio tabla_borrar.dict_medio],
    label=["Vector" "Dict"],
    xlabel="Tamaño n",
    ylabel="Tiempo medio (ns)",
    title="Tiempo de borrar!",
    marker=:circle,

```

```

linewidth=2,
xscale=:log2,
yscale=:log10
)

```



```

for fila in eachrow(tabla_borrar)
    println("n=$(fila.n): gana $(fila.comparar)")
end

```

```

n=4: gana Dict
n=8: gana Dict
n=16: gana Dict
n=32: gana Dict
n=64: gana Dict
n=128: gana Dict
n=256: gana Dict
n=512: gana Dict
n=1024: gana Dict
n=2048: gana Dict
n=4096: gana Dict

```

2.3.2 Análisis de borrar!

Pendiente analisis

2.4 Experimento: insertar!

Para la inserción se utiliza la llave $n + 1$, que no se encuentra entre las llaves iniciales en el caso del diccionario. En el caso del vector, se utiliza una capacidad de n dado que fue la instrucción de la actividad. Esto permite que la inserción se realice en la primera posición vacía.

```
resultados_insertar_vector = []
resultados_insertar_dict = []

for n in tamaños
    v, d = preparar_datos(n, n + 1)

    sizehint!(d, n + 1) # Para el diccionario

    resultado_vector = @benchmark insertar_vector!(v, nueva_llave, 1.0) setup=(
        v = copy($v);
        nueva_llave = Int64($n + 1)
    ) samples=100 evals=1

    resultado_dict = @benchmark insertar_dict!(d, nueva_llave, 1.0) setup=(
        d = copy($d);
        nueva_llave = Int64($n + 1)
    ) samples=100 evals=1

    push!(resultados_insertar_vector,
        (
            n = n,
            trial = resultado_vector,
            minimo = minimum(resultado_vector).time,
            medio = mean(resultado_vector).time,
            maximo = maximum(resultado_vector).time
        )
    )

    push!(resultados_insertar_dict,
        (
            n = n,
```

```

        trial = resultado_dict,
        minimo = minimum(resultado_dict).time,
        medio = mean(resultado_dict).time,
        maximo = maximum(resultado_dict).time
    )
)
end

```

2.4.1 Resultados de insertar!

```

tabla_insertar = DataFrame(
    n = [r.n for r in resultados_insertar_vector],
    vector_minimo = [r.minimo for r in resultados_insertar_vector],
    vector_medio = [r.medio for r in resultados_insertar_vector],
    vector_maximo = [r.maximo for r in resultados_insertar_vector],
    dict_minimo = [r.minimo for r in resultados_insertar_dict],
    dict_medio = [r.medio for r in resultados_insertar_dict],
    dict_maximo = [r.maximo for r in resultados_insertar_dict]
)

tabla_insertar.comparar = ifelse.(
    tabla_insertar.vector_medio .< tabla_insertar.dict_medio,
    "Vector",
    "Dict"
)

tabla_insertar

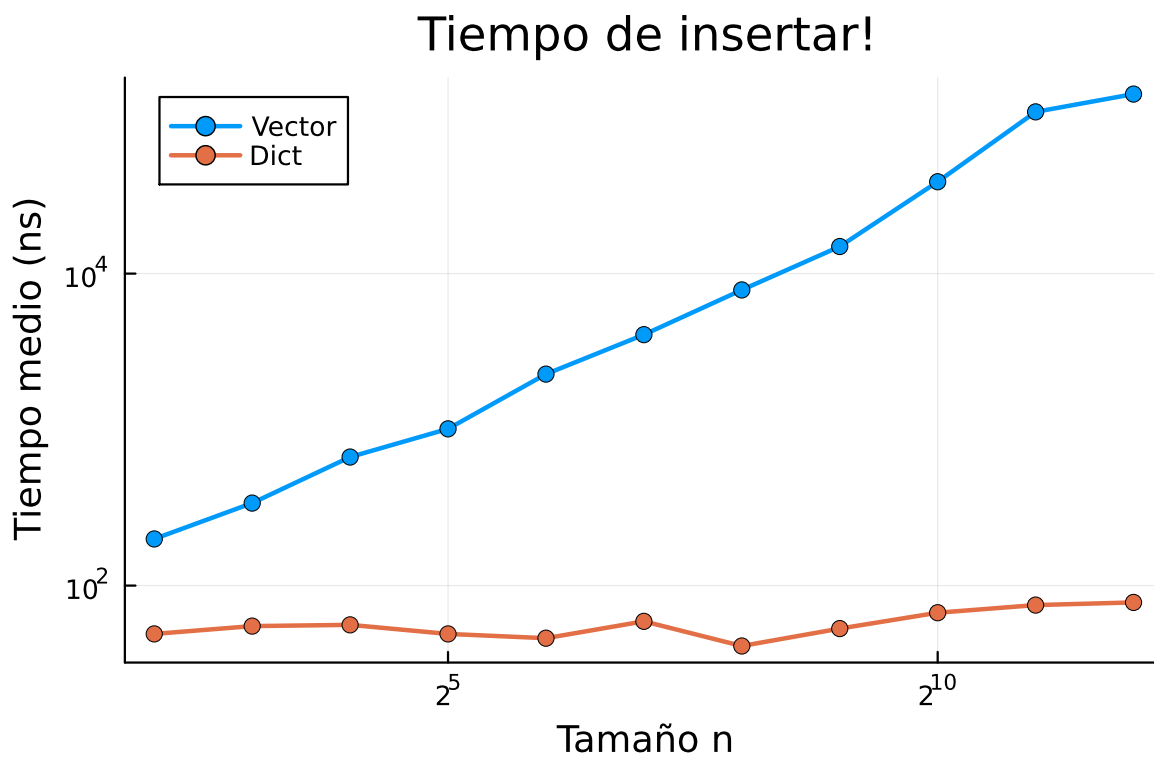
```

	n	vector_minimo	vector_medio	vector_maximo	dict_minimo	dict_medio	dict_maximo
	Int64	Float64	Float64	Float64	Float64	Float64	Float64
1	4	100.0	199.0	3800.0	0.001	49.0005	100.0
2	8	300.0	338.0	3500.0	0.001	55.0005	100.0
3	16	600.0	667.0	4600.0	0.001	56.0004	100.0
4	32	900.0	1011.0	5200.0	0.001	49.0005	100.0
5	64	1900.0	2269.0	9000.0	0.001	46.0005	100.0
6	128	3700.0	4060.0	12400.0	0.001	59.0004	100.0
7	256	7500.0	7858.0	21700.0	0.001	41.0006	100.0
8	512	14000.0	14916.0	32800.0	0.001	53.0005	200.0
9	1024	36000.0	38806.0	82700.0	0.001	67.0004	400.0
10	2048	70300.0	108826.0	1.4499e6	0.001	75.0003	300.0
11	4096	131300.0	141574.0	236600.0	0.001	78.0002	300.0

```

plot(
  tabla_insertar.n,
  [tabla_insertar.vector_medio tabla_insertar.dict_medio],
  label=["Vector" "Dict"],
  xlabel="Tamaño n",
  ylabel="Tiempo medio (ns)",
  title="Tiempo de insertar!",
  marker=:circle,
  linewidth=2,
  xscale=:log2,
  yscale=:log10
)

```



```

for fila in eachrow(tabla_insertar)
  println("n=$(fila.n): gana $(fila.comparar)")
end

```

```

n=4: gana Dict
n=8: gana Dict
n=16: gana Dict

```

```
n=32: gana Dict
n=64: gana Dict
n=128: gana Dict
n=256: gana Dict
n=512: gana Dict
n=1024: gana Dict
n=2048: gana Dict
n=4096: gana Dict
```

2.4.2 Análisis de insertar!

Pendiente el analisis de insertar

3 Comparación de rangos

Usando la columna `comparar` de cada tabla determinamos el menor tiempo medio en cada tamaño.

```
function mostrar_rangos(tabla)
    tamaños_vector = [fila.n for fila in eachrow(tabla) if fila.comparar == "Vector"]
    tamaños_dict = [fila.n for fila in eachrow(tabla) if fila.comparar == "Dict"]

    println("Vector: ", tamaños_vector)
    println("Dict: ", tamaños_dict)
end

println("Buscar")
mostrar_rangos(tabla_buscar)

println("Borrar")
mostrar_rangos(tabla_borrar)

println("Insertar")
mostrar_rangos(tabla_insertar)
```

```
Buscar
Vector: Any[]
Dict: [4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096]
Borrar
Vector: Any[]
```

Dict: [4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096]
Insertar
Vector: Any[]
Dict: [4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096]

4 Discusión

Pendiente

5 Conclusiones

Pendiente

6 Referencias

Cormen, Thomas H.; Leiserson, Charles E.; Rivest, Ronald L.; Stein, Clifford (2022). Introduction to Algorithms (2nd ed.). MIT Press.

Pendiente georeferenciar: <https://sadit.github.io/ALGO-IR/cap2-analisis.html>

Pendiente georeferenciar: <https://juliaci.github.io/BenchmarkTools.jl/stable/>

Pendiente georeferenciar: <https://docs.julialang.org/en/v1/base/collections/>

Aho, A. V., Hopcroft, J. E., & Ullman, J. D. (1983). Data structures and algorithms. Addison-Wesley.

“Big-O.” An Open Guide to Data Structures and Algorithms. Enfoque introductorio con definición formal y ejemplos.